Введение в API памяти

В этой лекции мы рассмотрим какие существуют инструменты выделения памяти в UNIX-системах.

Суть проблемы: Как выделять и управлять памятью?
В UNIX/C программах понимание того как выделять и управлять памятью является критичным для построения надежного ПО. Какие интерфейсы обычно используются? Какие ошибки следует избежать?

Типы памяти

При написании программ на C существует два типа выделяемой памяти. Первая - называется стеком и его алокацией и деалокацией занимается собственно сам компилятор. Поэтому иногда это называется **автоматической памятью**.

Декларирование памяти в стеке на языке C довольно просто. Например, для выделения памяти для некоторой переменной в некоторой функции делается следующим образом:

```
void func() {
    int x; // declares an integer on the stack
    ...
}
```

Компилятор сделает все остальное, убедится в том, что когда вы вызовете функцию func(), в стеке выделится место под переменную. Когда происходит возврат из функции, компилятор деалоцирует память, таким образом, если есть необходимость сохранять переменную и после возврата из функции, лучше для этого не пользоваться стеком.

Для того чтобы долго хранить в памяти какие-либо данные существует другой тип памяти, называемый кучей, в которой все алокации и деалокации осуществляются непосредственно программистом.

```
void func() {
    int *x = (int *) malloc(sizeof(int));
    ...
}
```

Вызов malloc()

Вызов malloc() довольно прост: ему нужно передать размер желаемой памяти. В качестве ответа он вернет указатель на кучу, или в случае ошибки вернет NULL. Для того чтобы использовать этот вызов нужно подключить заголовочный файл stdlib.h и все. В самом же вызове функции обычно не указывается конкретное число, используется какой-нибудь макрос, например sizeof(int)

```
double *d = (double *) malloc(sizeof(double));
```

Кроме того выделять память можно с помощью умножения результата функции на какую-то переменную или функцию, которая будет обсчитываться в момент вызова.

```
int *x = malloc(10 * sizeof(int));
printf("%d\n", sizeof(x));
```

Однако с этим нужно быть аккуратным, поскольку переменная может принимать разное значение. В данном примере x=4 для 32-битных систем и 8 для 64-битных, поскольку на самом деле здесь не возвращается размер выделенной памяти, здесь возвращается размер указателя.

Вызов free()

Выделение памяти в данном процессе - самая простая часть. А самая сложная - ее освобождение. Нужно подобрать время и способ для выполнения этой операции. Для освобождения неиспользуемой памяти кучи программисты обращаются к системному вызову free():

```
int *x = malloc(10 * sizeof(int));
...
free(x);
```

Операция требует всего одного аргумента - адреса указателя. Как вы могли заметить, размер выделенной области памяти не передается функции очистки, поскольку задача по отслеживанию занятых регионов памяти лежит на самих библиотеках выделения памяти.

Частые ошибки

Правильное управление памятью - одна из распространенных ошибок программистов. Хотя, некоторые более молодые языки чем C имеет автоматическое управление памятью. В таких языках вызывая malloc() или new для создания нового объекта вы никогда не очищаете память самостоятельно. За вас это будет делать так называемый **garbage collector**. Он запускается и просматривает выделенную память, выясняя, на какие участки памяти вы более не ссылаетесь и затем очищает их.

Следует понимать, что скомпилированная или даже запускающаяся программа != правильная программа. Даже если она отработала множество раз, могут быть какие-то ситуации, когда звезды складываются особым образом и какие-то неучтенные состояния могут нарушить процесс. Крайне редко проблемы встречаются в компиляторах и на низком уровне (процессоре или других железных компонентах), хотя и такое бывает.

1. Забыли выделить память

В некоторых случаях получается ситуация, когда к объекту обращаются до того как ожидаемая память была выделена. Например, выполняя функцию strcopy(dst, src), которая копирует строку с указателя-источника в указатель-назначение, если вы будете невнимательны, может произойти следующее:

```
char *src = "hello";
char *dst;          // oops! unallocated
strcpy(dst, src); // segfault and die
```

Этот код приведет к ошибке **segmentation fault (segfault).** Которая обычно означает следующее: “Вы сделали что-то неправильное с памятью, глупый вы программист и я зла на вас”.
Правильно в данном случае писать код вот так:

```
char *src = "hello";
char *dst = (char *) malloc(strlen(src) + 1);
strcpy(dst, src); // work properly
```

Альтернативно можно использовать strdup(), которая может немного облегчить вашу жизнь.

2. Выделили недостаточно памяти

Также частой ошибкой может являться ошибка выделения недостаточного количества памяти или переполнение памяти, еще называемая **buffer overflow.**

```
char *src = "hello";
char *dst = (char *) malloc(strlen(src)); // too small!
strcpy(dst, src); // work properly
```

Выглядит немного странно, но в зависимости от реализации malloc() и многих других деталей, эта программа чаще всего будет работать корректно. Но в некоторых случаях, когда выполняется копирование строки, она будет скопирована на 1 байт заходя за выделенное пространство. Иногда такой заступ может приводить к безобидным последствиям, например затирая неиспользуемую переменную, но иногда это может вылиться в нечто критичное. Большое количество уязвимостей в ПО построено именно на ошибке этого типа. Если даже это сделано неумышлено, в некоторых случаях вам может повезти и за выделенной областью памяти будет пусто, а в других случаях это может привести к сложно отлавливаемым ошибках или просто к краху программы. Отсюда и вывод, что даже если программа отработала корректно один раз, нет гарантии что она правильная.

3. Забыли инициализировать выделенную память

Даже при условии того что вы выделили память корректно, вы можете забыть заполнить ее данными. В этом случае можно столкнуться с ошибкой **неинициализированного чтения (uninitialized read).** В этот момент происходит попытка чтения из кучи некоторого неизвестного значения. Кто знает что там может быть? Если повезет, то может быть что-то вроде нуля, а вот в плохом случае может быть даже вредоносное.

4. Забыли освободить память

Еще одной ошибкой бывает ситуация, называемая **утечкой памяти (memory leak)**, которая случается когда вы забываете освобождать память. В приложениях, которые работают долго, например даже в самой ОС это может быть крупной проблемой, так как по мере работы ПО память может утекать, что будет приводить к нехватки памяти в системе и в этой ситуации поможет только перезагрузка. Поэтому, когда вы заканчиваете работать с кусочком памяти следует выполнять его очистку. И следует помнить, что использование сборщика мусора не поможет в этой ситуации, если есть какие-либо указатели на адрес в куче, сборщик мусора не станет очищать эту память.

В некоторых ситуациях вызов free() может быть излишним. В ОС существует 2 уровня управления памятью. Первый уровень управляется самой ОС, на втором - программой в пределах ее адресного пространства, в которых она может выполнять malloc(), free(). Если программа работает недолго и скоро будет завершена, то при выходе ОС сама очистит всю память за процессом и в этом случае утечка будет сильно незаметна, близка к нулю. Таким образом ОС убеждается что никакие участки памяти не будут потеряны. Однако, хотя такой подход и работает, подобная привычка довольно плоха и вырабатывает неправильный стиль программирования. И хотя вы можете и не делать этого, все равно стоит очищать каждый выделенный байт памяти.

5. Очистка памяти до завершения работы с ней

Иногда память программы очищается до того как она заканчивает работу с ней. Такая ошибка называется **висячий указатель (dangling pointer).** Результат этого непредсказуем. Библиотека работы с памятью при обращении к ней в таком случае может выкинуть неизвестный финт, чаще всего это приводит к падению или к перезаписи нормальной памяти (например вызвали free(), затем malloc() который перезатрет память которая еще должна была использоваться программой).

6. Циклическая очистка памяти

Программа иногда может очищать память более одного раза. Эта ошибка называется **двойная очистка (double free).** Результат также может быть непредсказуем. Библиотека

работы с памятью может также выкинуть непонятный финт. Частый исход - крах программы.

7. Некорректный вызов free()

Последней проблемой, которую мы обсудим, это неправильный вызов free(). Этот вызов рассчитывает в качестве входящего параметра только адрес в памяти, который ранее получен в качестве возврата от malloc(). Если передать ему какое-либо другое значение, то может случиться неизвестно что. Эта ошибка называется **неправильная очистка (invalid free).** Такие ошибки опасны и их следует избегать.

Итоги

Как вы могли заметить существует множество способов некорректного обращения с памятью. Из-за подобных частых ошибок была разработана целая экосистем утилит для поиска ошибок в памяти Например это valgrind и purify.

Низлежащая поддержка ОС

Мы не обсуждали системные вызовы в этой лекции, поскольку ни malloc() ни free() такими не являются - они библиотечные вызовы. Они управляют памятью в пределах адресного пространства, но сами эти вызовы внутри построены на нескольких системных вызовах к ОС, которые уже и выделяют или возвращают неиспользуемую память обратно в систему.

Один из таких выводов называется brk, который нужен для того чтобы изменять конец кучи в программе. Он принимает один аргумент (адрес нового конца) и увеличивает или уменьшает размер кучи, в зависимости от переданного значения (оно больше или меньше текущего). Напрямую вызывать brk\sbrk не нужно, они должны быть использованы через библиотеки управления памятью - free, malloc.

Память также можно получить через системный вызов mmap(), передав ей корректные аргументы. Эта функция **выделяет анонимную память (anonymous memory)** в пределах программы - такой регион памяти, который не ассоциируется ни с одним файлом, но ассоциируется с **пространством swap**. Такая память также может быть использована как и обычная память, получаемая с помощью malloc().

Кроме того существуют еще некоторые другие системные вызовы, такие как calloc(), который выделяет память и заполняет ее нулями перед тем как освободить. Такой способ может частично помочь с ошибками типа неинициализированного чтения. Вызов realloc() помогает в случае когда например есть какой-то массив, в который требуется что-то добавить, вызывая realloc() мы получаем более крупный регион памяти со скопированными значениями предыдущего региона. В качестве возврата отдает указатель на новый регион.